

TP 1 — Environnement, Git et les limites du traitement local

Big Data Engineering — Master 1 — DMI/FST/UCAD — Prof. Samba Ndiaye

Ce notebook guide les parties **C** (génération des données), **D** (exploration Pandas) et **E** (montée en charge) du TP 1. Complétez les cellules marquées **À COMPLÉTER**, exécutez tout de bout en bout, puis poussez le notebook **avec ses sorties** dans `notebooks/` de votre dépôt Git.

Réflexe d'ingénieur : on ne dit pas « c'est lent », on dit « 12,4 s pour 50 000 lignes ». **Toutes** vos observations doivent être chiffrées.

0. Vérification de l'environnement

La cellule suivante doit s'exécuter sans erreur et afficher des versions cohérentes (Python ≥ 3.9). Sinon, retournez à la partie A du TP (`python3 check_env.py`).

```
In [ ]: import sys, platform
import pandas as pd
import numpy as np

print("Python :", sys.version.split()[0], "-", platform.system())
print("pandas :", pd.__version__)
print("numpy  :", np.__version__)
```

1. Génération du jeu de données fil rouge (échelle 0.1)

La génération est **déterministe** (graine 42) : toute la promotion obtient exactement les mêmes fichiers. Exécutez la cellule ci-dessous **une seule fois** (≈ 1 min). Le dossier `data/` est exclu du dépôt Git par le `.gitignore` fourni — vérifiez-le avec `git status`.

```
In [ ]: # Depuis la racine du dépôt (adapter Le chemin si besoin)
!python ../data/generate_data.py --scale 0.1 --outdir ../data
!dir ../data
```

Tableau de relevés

Vous remplirez ce dictionnaire au fil des exercices ; il sert de source unique pour la partie E et pour votre `DIAGNOSTIC.md`.

```
In [ ]: DATA_DIR = "../data"

releves = {
```

```
# fichier : {"lignes": ..., "temps_s": ..., "memoire_Mo": ..., "disque_Mo": ...}
"customers.csv" : {"lignes": 5000 },
"orders.csv" : {"lignes": 50000},
"orders+items (jointure)" : {},
"events.json" : {"lignes": 330000 },
}
```

Exercice 1 — customers.csv : premier contact

Chargez le fichier, puis relevez :

1. ses **dimensions** (lignes × colonnes) ;
2. le **type** de chaque colonne (`dtypes`) ;
3. un **aperçu** (`head`) ;
4. son **empreinte mémoire** en Mo — utilisez `df.memory_usage(deep=True).sum()` (pourquoi `deep=True` ? voyez la question 1.b).

```
In [ ]: # === À COMPLÉTER ===
import os, time

DATA_DIR = os.path.join("../", "data")

t0 = time.perf_counter()
customers = pd.read_csv(os.path.join(r"C:/Users/24kha/Desktop/MSIA_ISI/TP BIG DATA", "customers.csv"))
t1 = time.perf_counter()

print("Dimensions :", customers.shape)
print("Temps de chargement :", round(t1 - t0, 2), "s")
```

Question 1.a — Remplissez la ligne `customers.csv` du tableau de relevés.

```
In [ ]: # === À COMPLÉTER ===
mem_Mo = customers.memory_usage(deep=True).sum() / (1024 ** 2) # octets -> Mo
disque_Mo = os.path.getsize(os.path.join(DATA_DIR, "customers.csv")) / (1024 ** 2)

relevés["customers.csv"] = {
    "lignes": len(customers),
    "temps_s": round(t1 - t0, 2),
    "memoire_Mo": round(mem_Mo, 1),
    "disque_Mo": round(disque_Mo, 1)
}
relevés["customers.csv"]
```

Question 1.b (markdown — répondez ici) — Comparez `memory_usage()` **sans** puis **avec** `deep=True` sur `customers`. Pourquoi un tel écart ? Quel est le rapport mémoire/disque ? Notez votre explication : elle resservira mot pour mot dans le diagnostic.

Votre réponse : Sans `deep=True`, `memory_usage()` ne compte que la mémoire des buffers de données principaux (index et colonnes), donc elle sous-estime la place réellement occupée par les chaînes de caractères et les objets Python. Avec `deep=True`, pandas parcourt chaque élément et compte aussi la mémoire des objets Python stockés

dans les colonnes textuelles, d'où un écart important. Sur `customers`, ce surcoût est visible parce que plusieurs colonnes sont textuelles. En pratique, la taille mémoire en mémoire vive est de l'ordre de 2 à 4 fois la taille du fichier CSV sur disque : le CSV est stocké de façon compacte en texte, alors que pandas charge des structures plus lourdes avec des objets Python et des métadonnées supplémentaires.

Exercice 2 — `orders.csv` : statistiques et première surprise

Chargez `orders.csv` en **chronométrant**, puis :

1. nombre de commandes par `statut` ;
2. nombre de commandes par `canal` ;
3. essayez de calculer le **montant total** des commandes (`montant_total_fcfa`)... que se passe-t-il ? Regardez le `dtype` de la colonne. **Ne corrigez pas** : documentez (section 6).

```
In [ ]: # === À COMPLÉTER ===
t0 = time.perf_counter()
orders = pd.read_csv(os.path.join(DATA_DIR, "orders.csv")) # charger DATA_DIR/
t1 = time.perf_counter()
print("Temps : %.2f s - %d lignes" % (t1 - t0, len(orders)))

print(orders["statut"].value_counts()) # commande
print(orders["canal"].value_counts(normalize=True).round(3)) # commande

# tentative de somme :
print(orders["montant_total_fcfa"].dtype)
# -> object : ~1 % des montants portent un suffixe " FCFA", la colonne entière
# est donc lue comme chaîne. La somme directe échoue ou concatène.
try:
    print(orders["montant_total_fcfa"].sum())
except Exception as e:
    print("Erreur attendue :", type(e).__name__, e)
```

Question 2.a — Remplissez la ligne `orders.csv` du tableau de relevés (lignes, temps, mémoire deep, taille disque).

```
In [ ]: # === À COMPLÉTER ===
relevés["orders.csv"] = {
    "lignes": len(orders),
    "temps_s": round(t1 - t0, 2),
    "memoire_Mo": round(orders.memory_usage(deep=True).sum() / 1e6, 1),
    "disque_Mo": round(os.path.getsize(os.path.join(DATA_DIR, "orders.csv")) / 1e6, 1)
}
relevés["orders.csv"]
```

Exercice 3 — Jointure `orders` × `order_items`

Les jointures sont au cœur de l'analytique — et elles **coûtent cher**.

1. Chargez `order_items.csv`.
2. Mesurez la mémoire totale **avant** la jointure (somme des deux DataFrames).
3. Réalisez la jointure (`merge` sur `order_id`), chronométrez-la.
4. Mesurez la mémoire du résultat. Conclusion ?

```
In [ ]: # === À COMPLÉTER ===
items = pd.read_csv(os.path.join(DATA_DIR, "order_items.csv")) # charger order
mem_avant = (orders.memory_usage(deep=True).sum()
             + items.memory_usage(deep=True).sum()) / 1e6 #

t0 = time.perf_counter()
joint = orders.merge(items, on="order_id", how="inner") #
t1 = time.perf_counter()

mem_joint = joint.memory_usage(deep=True).sum() / 1e6
print("Jointure : %.2f s - %d lignes" % (t1 - t0, len(joint)))
print("Mémoire avant : %.0f Mo - résultat seul : %.0f Mo" % (mem_avant, mem_joint))
# Observation : Le résultat duplique les colonnes de la commande sur CHAQUE
# ligne d'article -> la mémoire du résultat dépasse la somme des entrées.
# Pendant le merge, entrées + sortie coexistent : pic mémoire ~ 2-3x.

relevés["orders+items (jointure)"] = {
    "lignes": len(joint), "temps_s": round(t1 - t0, 2),
    "memoire_Mo": round(mem_joint, 1), "disque_Mo": None,
}
```

Exercice 4 — `events.json` : le gros morceau

`events.json` est au format **JSON Lines** (un objet JSON par ligne). À l'échelle 0.1 il reste chargeable (~330 000 lignes) ; mesurez précisément : temps, mémoire deep, taille disque, et le **ratio mémoire/disque**.

```
In [ ]: path_ev = os.path.join(DATA_DIR, "events.json")
t0 = time.perf_counter()
events = pd.read_json(path_ev, lines=True) # pd.read_json
t1 = time.perf_counter()

mem_Mo = events.memory_usage(deep=True).sum() / 1e6
disque_Mo = os.path.getsize(path_ev) / 1e6
relevés["events.json"] = {
    "lignes": len(events), # ~330 000 à l'échelle 0.1
    "temps_s": round(t1 - t0, 2),
    "memoire_Mo": round(mem_Mo, 1),
    "disque_Mo": round(disque_Mo, 1),
} # relevés
print("Ratio mémoire/disque : %.1f x" % (mem_Mo / disque_Mo))
# Ordre de grandeur attendu : 2 à 5x (chaînes -> objets Python, index, etc.)
relevés["events.json"]
```

Exercice 5 — Synthèse des relevés (échelle 0.1)

```
In [ ]: pd.DataFrame(relevés).T
```

Partie E — Montée en charge : échelle 1.0 sur Colab

À faire sur Google Colab (pas sur votre laptop) :

1. Téléversez `generate_data.py` et `requirements.txt` dans la session Colab ;
2. Installez les dépendances avec `!pip install -r requirements.txt`, puis générez les données avec `!python3 generate_data.py --scale 1.0 --outdir ./data` (environ 2 minutes, environ 860 Mo de données) ;
3. Ré-exécutez les exercices 2 à 4 sur ces nouvelles données en adaptant `DATA_DIR` à `./data` ;
4. Tentez le chargement intégral de `events.json` (environ 685 Mo, 3,3 millions de lignes) et **notez** : la durée, l'occupation de la RAM dans l'onglet « RAM » de Colab, les avertissements éventuels et tout crash du noyau.

Un crash du noyau **est un résultat de mesure**, pas un échec : notez précisément ce qui s'est passé et à quel moment.

```
In [ ]: # Cellule Colab (échelle 1.0) – recopiez vos mesures ci-dessous en Local
releves_colab = {
    "orders.csv (1.0)" : {"lignes": None, "temps_s": None, "memoire_Mo": None},
    "events.json (1.0)" : {"lignes": None, "temps_s": None, "memoire_Mo": None,
                           "issue": "chargé / averti / crash ?"},
}
releves_colab
```

Extrapolation

Complétez à partir de vos deux points de mesure (0.1 et 1.0), en supposant d'abord une croissance **linéaire** :

Scénario	events.json	Temps estimé	Mémoire estimée	Tenable sur 1 machine ?
TP échelle 0.1 (mesuré)	≈ 70 Mo	faible	faible	Oui
TP échelle 1.0 (mesuré)	≈ 685 Mo	élevé	élevé	Limité, mais encore possible sur une machine puissante
Plateforme réelle, 1 an	≈ 50 Go	environ 70 à 100 fois plus élevé que l'échelle 1.0	environ 70 à 100 fois plus élevé	Non, pas sur une seule machine classique
Grand acteur régional	≈ 1 To	très supérieur à la capacité d'un seul poste	très supérieur à la capacité d'un seul poste	Non, il faut du scale-out

Réponses détaillées :

1. **L'extrapolation linéaire est-elle optimiste ou pessimiste ?** Elle est plutôt optimiste. Entre 0.1 et 1.0, le volume augmente d'environ 10 fois, mais le coût réel de lecture, de mémoire et de calcul augmente souvent plus vite que cela à cause des jointures, de l'overhead Python, du stockage des chaînes de caractères, de la fragmentation mémoire et des coûts de gestion du système. En pratique, on atteint très vite un mur de mémoire et de temps.
2. **Quelles parades sans distribution connaissez-vous ?** On peut utiliser des chunks pour lire par morceaux, des `dtype` plus compacts (par exemple numériques au lieu de chaînes), l'échantillonnage pour l'exploration, et des formats binaires comme Parquet ou Feather. On peut aussi filtrer tôt, ne garder que les colonnes utiles et éviter de charger des objets lourds en mémoire. Ces méthodes repoussent le mur, mais elles ne suffisent pas au-delà de quelques dizaines de Go à quelques centaines de Go sur une seule machine.
3. **À partir de quel point le scale-out devient-il inévitable ?** Dès que les données ne tiennent plus en RAM, que les temps de chargement deviennent trop longs, ou que les traitements impliquent des jointures et des agrégations trop coûteuses pour une seule machine, il faut passer à une architecture distribuée. Sur un poste standard, cela se produit rapidement dès qu'on dépasse quelques dizaines de Go de données traitées ou quand le dataframe devient trop lourd à manipuler complètement en mémoire.

6. Anomalies repérées (sans les corriger !)

Listez ici, **avec une preuve chiffrée** (un comptage, un exemple), les bizarreries rencontrées. Elles seront traitées en séances 3 et 9.

Exemples concrets à documenter :

- Dans `orders.csv`, la colonne `montant_total_fcfa` n'est pas de type numérique : elle est lue comme `object` car certains montants contiennent un suffixe `FCFA`, ce qui empêche une somme directe.
- Dans `customers.csv`, plusieurs valeurs textuelles peuvent contenir des cas douteux (emails vides ou `N/A`, villes avec casse/accents incohérents, etc.), ce qui est typique des données brutes à nettoyer.
- Les dates de naissance ou autres champs de type texte peuvent contenir des valeurs aberrantes ou mal formées, à vérifier par comptage et à corriger plus tard.
- Les colonnes textuelles sont coûteuses en mémoire, ce qui explique déjà l'écart important entre la taille du CSV et l'empreinte mémoire chargée dans pandas.

```
In [ ]: import os
import pandas as pd

customers = pd.read_csv(os.path.join(DATA_DIR, "customers.csv"))
orders = pd.read_csv(os.path.join(DATA_DIR, "orders.csv"))

# 1) Montant total lu comme chaîne : anomalie de type
print("dtype montant_total_fcfa :", orders["montant_total_fcfa"].dtype)
print("Exemples :", orders["montant_total_fcfa"].head(10).tolist())
```

```
# 2) Emails vides ou manquants
print("Emails manquants :", customers["email"].isna().sum())
print("Emails vides :", (customers["email"].astype(str).str.strip() == "").sum())

# 3) Variantes de casse dans les villes
print(
    "Villes avec casse différente :",
    customers["ville"].str.upper().duplicated().sum(),
    "lignes potentiellement dupliquées en casse"
)
print("Exemples de villes :", customers["ville"].head(10).tolist())

# 4) Vérification des dates de naissance
try:
    dates = pd.to_datetime(customers["date_naissance"], errors="coerce")
    print("Dates invalides :", dates.isna().sum())
except Exception as e:
    print("Erreur de conversion des dates :", e)
```

7. Vers le diagnostic

Avant de fermer ce notebook, vérifiez que vous savez répondre, **mesures à l'appui**, aux trois questions du canevas `docs/DIAGNOSTIC.md` :

1. **Constats** — où, précisément, le traitement local a-t-il atteint ses limites (chiffres) ?
2. **Analyse** — pourquoi (mémoire vs disque, jointures, croissance des volumes) ?
3. **Besoins** — qu'attend-on d'une architecture distribuée (et que garde-t-on de Pandas) ?

Puis: `git add notebooks/ docs/ && git commit -m "TP1 : exploration et diagnostic" && git push`.